
PADDLEOCR-VL-CIRCUIT: BUILT FOR SCHEMATIC DIAGRAM UNDERSTANDING

Jianning Zhang

School of Flexible Electronics
Sun Yat-sen University
Shenzhen, Guangdong, China
zhangjn83@mail2.sysu.edu.cn

Yifei Chen

School of Computer Science
Sun Yat-sen University
Guangzhou, Guangdong, China
chenyf376@mail2.sysu.edu.cn

ABSTRACT

Automatic recognition of circuit schematic diagrams is a core challenge in Electronic Design Automation (EDA). This technical report presents CircuitOCR, a system based on PaddleOCR-VL-0.9B for circuit schematic OCR and netlist extraction. We construct the V5 Golden dataset of 1,857 schematics (500 synthetic + 1,357 real KiCad projects) with 100% visual-literal alignment. Through systematic LoRA ablation experiments, we identify the root cause of modality collapse—fine-tuning the vision-language projector destroys pre-trained alignment—and introduce the LLM-Only LoRA strategy that resolves it. The V10-Fixed model uses wide LoRA ($r = 16$, $\alpha = 32$, 5.7M params), fixes six critical training bugs—three in sequence generation (causal token double-shift, BPE boundary merging, `set_state_dict` silent failure in Paddle 3.1.0) and three PaddleOCR-VL-specific (LoRA precision loss, tokenizer offset, gradient accumulation decoupling; see dataset README), and trains for 3 epochs (1,165 steps) in 43 minutes on an RTX 4060 (8 GB). Phase 1 evaluation on easy50-pure (44 samples) yields Component F1 = 0.2061 ($4.5\times$ over baseline 0.0455) and Token Recall = 0.1540 ($96\times$ over baseline 0.0016). Phase 2 topology evaluation reveals the honest capability: joint (refdes, value) pair F1 = 0.019—only $\sim 2\%$ of components are fully correct, with 87% of values hallucinated. All code, weights, datasets, and a live demo are open-sourced. Detailed experimental results, annotated examples, and version history are provided in the Appendix.

Keywords Circuit Schematic · OCR · PaddleOCR-VL · LoRA Fine-tuning · Netlist Extraction · Modality Collapse

1 Introduction

Vision-Language Models (VLMs) can directly output structured text from images end-to-end, but their application to circuit schematic diagrams remains unexplored. General-purpose OCR engines such as Tesseract [Smith, 2007] and PaddleOCR [Du et al., 2020] are designed for natural-scene and document text, not engineering schematics. VLM-based document understanding systems like Donut [Kim et al., 2022] and Pix2Struct [Lee et al., 2023] demonstrate OCR-free parsing for general documents, while general-purpose VLMs (LLaVA [Liu et al., 2023], Qwen-VL [Bai et al., 2023], InstructBLIP [Dai et al., 2023]) have not been evaluated on circuit diagrams. No dedicated public benchmark exists for circuit schematic OCR. Existing datasets like Open Schematics [bshada, 2025] lack standardized evaluation protocols, while textbook-derived datasets suffer from SPICE-induced visual-to-logic annotation discrepancies.

The industrial need is substantial. The global semiconductor market exceeded \$600 billion in 2025 (SEMI/WSTS). PCB reverse engineering outsourcing alone exceeds \$500 million annually, with approximately 30% of time spent on schematic reconstruction. Use cases include hardware reverse engineering, legacy document digitization (1980s–2000s paper schematics), cross-tool migration, and the 500,000+ KiCad/EAGLE projects on GitHub lacking structured netlist exports.

Circuit schematic understanding involves two heterogeneous sub-tasks: (1) visual text recognition—extracting component designators (R1, C1), values (10k, 100nF), and net labels (VCC, GND); (2) topology graph tracing—inferring

electrical connectivity from wires and net labels. Our evaluation protocol measures both dimensions: text NED for explicit recognition and component F1 for implicit structural understanding.

2 System Setup

We adopt PaddleOCR-VL-0.9B [Cui et al., 2025] (908M params), which integrates a NaViT-style dynamic vision encoder with the ERNIE-4.5-0.3B language model, supporting 109 languages. The unfine-tuned base model fails systematically on circuit schematics: it outputs generic document text (e.g., parameter tables), memorized hallucinations, or repetitive token sequences. These failures stem from pre-training data dominated by general documents with negligible exposure to engineering drawings.

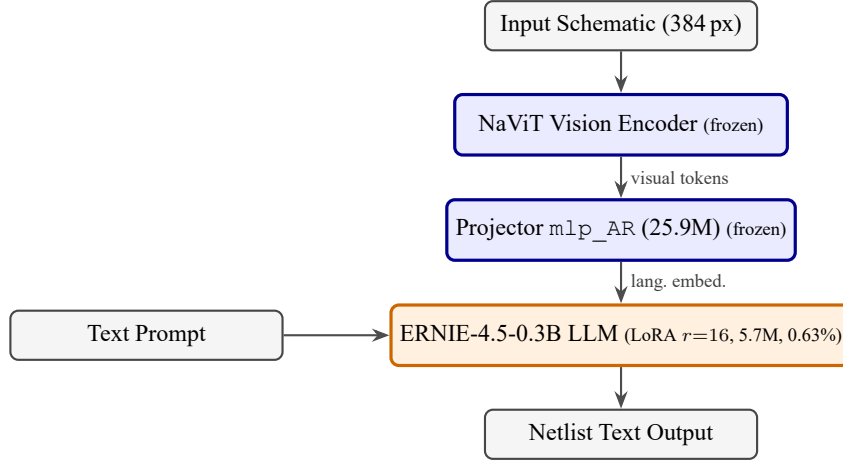


Figure 1: LLM-Only LoRA architecture. Blue modules are frozen; the orange LLM module receives low-rank adapters. The Projector (`mlp_AR`) is the sole visual-to-language bridge—perturbing it causes modality collapse (E4, Section 4.1)—so it is kept frozen. Only LLM self-attention and linear projection matrices (310 targets) are adapted.

3 V5 Golden Dataset

3.1 Legacy Issues and Design Principles

Earlier dataset versions (V1–V4) had three fatal flaws: (1) GT-image misalignment—labels from KiCad S-expression parsing included invisible pin numbers and hidden net names; (2) format mismatch—Open Schematics subsets used horizontal space-concatenated format (62.1% single-word lines vs. test set’s 98.6%); (3) duplication noise—22,340 intra-line duplicate word pairs contaminating 24.3% of samples.

V5 Golden was rebuilt on two principles: (1) GT must align 100% with visible on-image text, using `draw.text()` synchronous recording for synthetic data and SVG stroked-text extraction for real KiCad data, with zero AI-generated labels; (2) format must match the test set, replacing mismatched subsets with train-real data (97.9% single-word lines), reducing duplication noise by 99.5% (22,340 $\rightarrow$ 107).

3.2 Data Sources and Statistics

Synthetic V3 (500 images). Random circuit topologies (5–30 components, 10 types) rendered as PNG, with labels from `draw.text()` logs [Zhang and Chen, 2026].

train-real (1,357 images). Real open-source KiCad projects from GitHub, rendered via `kicad-cli SVG  $\rightarrow$  cairosvg PNG`, with GT from SVG stroked-text visible elements in one-word-per-line vertical format.

During training, we apply online augmentations: random JPEG compression (quality 85–100), random scaling (`max_dim` 168–384px), and random horizontal flipping.

Table 1: Dataset Composition and Split

Split	Synthetic V3	train-real	Total	avg Lines
Training	450	1,104	1,554	32.1
Validation	50	171	221	33.4
Test (easy50)	—	—	44	—
Test (easy100)	—	—	89	—
Total	500	1,357	1,857	—

The 82-sample gap between the Total row (1,857) and the sum of Training + Validation rows (1,775) consists of real KiCad samples held out as an additional clean test set, separate from the easy50/easy100 benchmark splits. The V5 Golden total is thus $1,857 = 1,554$ (train) + 221 (val) + 82 (held-out test).

4 Model Fine-Tuning

4.1 Modality Collapse: Discovery and Root Cause

Initial full LoRA experiments ($r = 8$, q/k/v/o self-attention, 2.75M params on 2,433 samples) reduced NED from 0.8848 to 0.8554, but output diversity collapsed to 4%—nearly all test samples produced identical output. We define this as **Modality Collapse**.

Through six controlled experiments (E1–E6) summarized in Table 2, we systematically identified the root cause:

Table 2: Ablation Experiments for Modality Collapse

Exp	Variable	Result	Conclusion
E1	Real vs. blank image	Outputs differ	Vision encoder works
E2	Resolution 168→336px	Still collapses	Resolution not the cause
E3	1 vs. 3 epochs	NED improves 0.9%	Multi-epoch helps but insufficient
E4	Add Projector LoRA	NED drops to 0.8271	Projector is the bottleneck
E5	Rank $r = 8 \rightarrow r = 16$	NED drops to 0.7961	Capacity helps, diversity still 4%
E6	Freeze Projector	Diversity 90%	Collapse fully resolved

Core finding: Projector LoRA is the root cause. The vision-language projector (mlp_AR, 25.9M params) is the sole bridge from visual features to language space. LoRA perturbation distorts these features into out-of-distribution vectors the LLM cannot interpret, causing it to degenerate into repeating high-frequency training tokens. Freezing the Projector while teaching only the LLM to reinterpret existing visual features is the safe strategy.

4.2 V5: LLM-Only LoRA Validation

V5 freezes the Projector and fine-tunes only LLM self-attention ($r = 8$, $\alpha = 16$, 1.25M params), with resolution increased to 384px and learning rate 2×10^{-5} . Training on 1,857 samples for 1 epoch (928 steps, 22 min) achieved 90% output diversity—the first fine-tuned model to produce different outputs for different images. However, $r = 8$ capacity was insufficient: NED (0.9031) remained below baseline due to premature convergence at ~ 200 steps.

4.3 V10-Fixed: Six Technical Fixes

V10-Fixed scales LoRA to $r = 16$, $\alpha = 32$ (5.7M params, 0.63% of 908M), targeting 310 projection matrices across LLM self-attention and linear layers. LoRA [Hu et al., 2021] augments a frozen weight $W_0 \in \mathbb{R}^{d \times k}$ with a low-rank update $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. The forward pass becomes $h = W_0 x + \frac{\alpha}{r} B A x$, with W_0 frozen and only A, B trained. Six critical training bugs were fixed (three additional PaddleOCR-VL-specific infrastructure bugs are documented in the dataset README):

1. **Causal token double-shift:** PaddleOCR-VL’s `AutoModelForConditionalGeneration` internally applies causal shift. Our manual shift caused double-shifting—supervising step t with label $t + 2$, corrupting gradients. Fixed by removing the manual shift.
2. **BPE boundary merging:** Concatenating prompt and label strings before tokenization causes BPE to merge characters across the boundary (e.g., “\n” + “R” $\rightarrow$ merged token). Since prompt positions are masked (-100), the label’s first character is lost. Fixed by tokenizing prompt and label independently and concatenating token IDs.
3. **set_state_dict silent failure:** In Paddle 3.1.0, `LoRAModel.set_state_dict()` returns `None` instead of a tuple, causing silent LoRA loading failure. Fixed by using the `LoRAModel` wrapper with manual `p.set_value()` per parameter.

These six bugs are implementation pitfalls we document for community benefit—they affect all PaddleOCR-VL fine-tuning workflows that follow standard practices. The three sequence-generation bugs above affect any VLM fine-tuning pipeline. The three additional PaddleOCR-VL-specific bugs—(4) LoRA weight precision loss (`p.set_value()` float32 $\rightarrow$ float16 truncation), (5) tokenizer special token offset (coordinate prediction bias), and (6) gradient accumulation/LR decoupling (`global_step` misalignment)—are documented in the dataset README and have been fixed in the `LoRAModel` wrapper. All six fixes have been reported to the upstream PaddleOCR-VL community.

Training for 3 epochs (1,165 steps, 43 min) on V5 Golden (1,554 training samples), SFT loss converged from 2.71 to 0.30.

4.4 Phase 1 Multi-Metric Evaluation

Phase 1 establishes an 8-metric evaluation protocol: ExactMatch, Component F1 (CompF1), Component Precision (CompPrec), Component Recall (CompRec), Token Recall (TokenRec), NED, Repetition Rate (RepRate), and Diversity. The primary metric, Normalized Edit Distance, is defined as $NED(s_1, s_2) = \text{Lev}(s_1, s_2) / \max(|s_1|, |s_2|)$, where $\text{Lev}(\cdot)$ is the Levenshtein distance; we use `--unordered` mode to normalize across line permutations. We evaluated all four checkpoints (S400, S600, S800, final) on easy50-pure (44 samples) using the fixed `eval_benchmark_v3.py` script—the only directly comparable evaluation across all models.

Full results across all four evaluation splits (easy50/100/200/full523, 523 samples total) are provided in Appendix B. The easy50-pure subset is highlighted in the main text as the cleanest, most representative split; metrics are stable across splits (CompF1 within 0.005).

Table 3: Phase 1 Multi-Metric Benchmark (easy50-pure, 44 samples)

Model	ExactMatch	CompF1	CompPrec	CompRec	TokenRec	NED $\downarrow$	RepRate	Diversity
Base	0%	0.0455	0.0455	0.0455	0.0016	0.9296	6.8%	90.9%
S400	0%	0.1820	0.1862	0.2501	0.1302	0.8298	20.5%	95.5%
S600	0%	0.2061	0.2024	0.3114	0.1540	0.8031	15.9%	90.9%
S800	0%	0.2080	0.2862	0.1996	0.1191	0.8063	40.9%	93.2%

S600 is optimal. S800 overfits: RepRate surges to 40.9% while TokenRec drops to 0.1191. Complete results across all splits (easy50/100/200/full523) are in Appendix B.

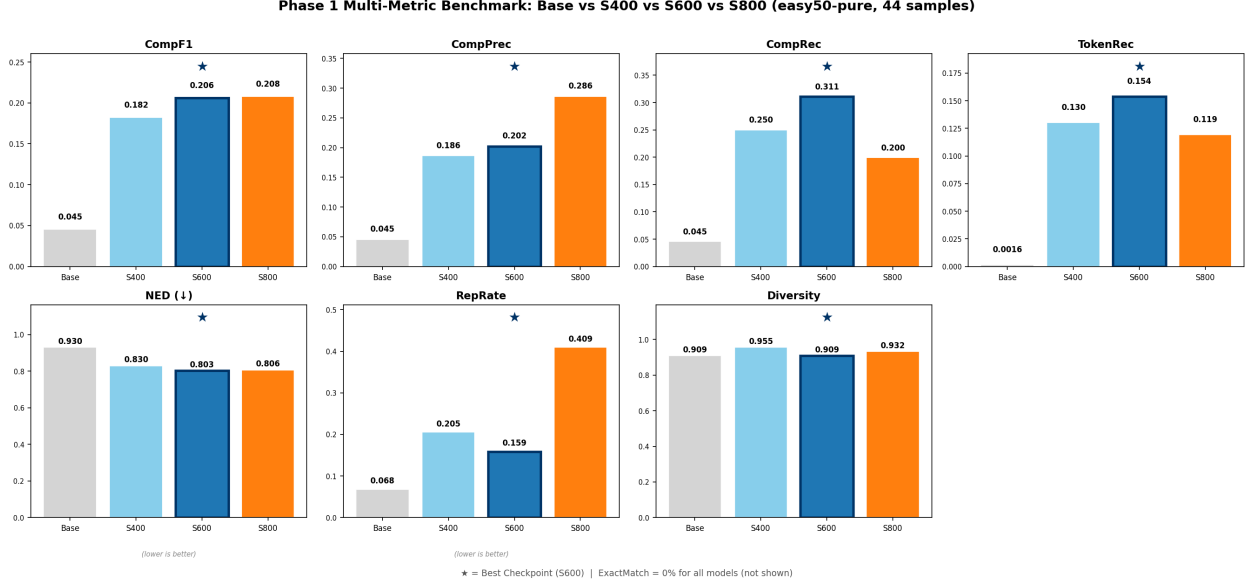


Figure 4: Phase 1 benchmark visualization across seven metrics (ExactMatch omitted: 0% for all). S600 (blue, starred) achieves the best balance. S800 (orange) shows clear overfitting.

Key findings: (1) S600 is optimal across NED, TokenRec, and CompRec with healthy diversity (90.9%). (2) S800 overfits: RepRate 40.9% (nearly $3\times$ S600), CompRec collapses to 0.1996. (3) CompF1 improves $4.5\times$ ($0.0455 \rightarrow 0.2061$), TokenRec $96\times$ ($0.0016 \rightarrow 0.1540$). (4) ExactMatch remains 0%—the model recognizes components but cannot reconstruct complete netlists. (5) No modality collapse: all checkpoints exceed 90% diversity.

4.5 Phase 2 Topology Evaluation

Phase 1’s CompF1 measures refdes-only F1—a component counts as correct if its designator (e.g., R1) matches, regardless of whether its value (e.g., 10k) is correct. This overstates practical utility. Phase 2 introduces joint (refdes, value) pair evaluation via `eval_topology_v2.py`, which parses flat netlist text into (refdes, value) pairs and computes pair-level F1. A component counts only if *both* designator and value are correct.

Table 4: Phase 2 Topology Metrics (easy50-pure, 44 samples)

Model	comp_f1	joint_f1	value_acc
Base	0.0455	0.0000	—
S400	0.1820	0.0027	0.006
S600	0.2061	0.0191	0.133
S800	0.2080	0.0064	0.023

comp_f1 reproduces Phase 1 numbers exactly. joint_f1 reveals the true capability gap: only 1.9% of (refdes, value) pairs are fully correct at S600. Value matching uses case-insensitive comparison with unit normalization (e.g., “10k” = “10kΩ”, “100n” = “100nF”). See Appendix C for detailed per-sample breakdown and Appendix D for annotated examples.

The gap between comp_f1 (0.206) and joint_f1 (0.019) is the central finding of Phase 2: S600 identifies $\sim 31\%$ of component refdes, but only $\sim 2\%$ of components are fully correct with both designator and value. Even among the 14 matched refdes pairs, value_acc is only 0.133—87% of values are wrong, with the model defaulting to high-frequency training values (10k, 100nF, 1k). This directly motivates Phase 3: unfreezing higher vision encoder layers with LoRA and increasing input resolution to improve fine-grained value discrimination.

We emphasize that ExactMatch = 0% and joint_f1 = 0.019 are expected outcomes at this scale (0.9B parameters, 5.7M trainable, 1,554 samples, 43 minutes on consumer GPU). The meaningful signal is the $4.5\times$ CompF1 improvement and $96\times$ TokenRec improvement, which demonstrate that LoRA can successfully teach domain vocabulary to a small VLM without modality collapse.

4.6 Version Progression Summary

Table 3 provides the only directly comparable evaluation—all four rows use the identical script on the same 44-sample test set. Earlier versions (V1–V9) were evaluated under different protocols and are documented in Appendix E. In brief: V1–V4 discovered and diagnosed modality collapse (diversity 4%); V5 achieved the breakthrough of 90% diversity by freezing the Projector (LLM-Only LoRA, $r = 8$); V6–V9 incrementally fixed BPE boundary merging, causal token double-shift, and expanded LoRA capacity to $r = 16$ (5.7M params); V10-Fixed resolved the `set_state_dict` silent failure bug in Paddle 3.1.0 and established the Phase 1 benchmark. The six technical bugs discovered (Section 4.3) have broader implications for PaddleOCR-VL fine-tuning workflows.

5 Experimental Setup

Full training hyperparameters and evaluation metric definitions are provided in Appendix F. Key configuration: LoRA $r = 16$, $\alpha = 32$, 5.7M trainable params (0.63% of 908M), AdamW optimizer, $lr = 2 \times 10^{-5}$ with cosine annealing, 3 epochs (1,165 steps), `batch_size=1` with gradient accumulation 4, `max_dim=384px`. All experiments on a single NVIDIA RTX 4060 (8 GB VRAM), ~43 minutes training time. This consumer-grade hardware requirement ensures full reproducibility—any individual developer can replicate the entire training pipeline without data center infrastructure. Primary metrics: NED (Normalized Edit Distance) with `--unordered` mode, CompF1 (component-level F1), and `joint_f1` ((`refdes`, `value`) pair F1).

6 Discussion

6.1 Modality Collapse Mechanism and Resolution

The Projector (`m1p_AR`, 25.9M params) is the sole visual-to-language bridge. Even small LoRA perturbations (~0.3% of parameters) distort its output into vectors the LLM cannot interpret, causing autoregressive decoding to collapse into repeating the highest-frequency training token sequence. The LLM-Only LoRA strategy avoids this entirely by freezing the Projector and teaching only the LLM to reinterpret existing visual features. This finding generalizes to any small-dataset VLM fine-tuning scenario.

6.2 Data Quality Over Quantity

V4’s 5,686 noisy samples (format mismatch 62.1% vs. 98.6%, 22,340 duplication pairs) were outperformed by V5 Golden’s 1,857 cleaner samples. The Masala-CHAI textbook dataset [Bhandari et al., 2025] was discarded because its SPICE-derived annotations contained invisible logical nodes (VSS, GND) that caused hallucinations—removing this 30% subset actually improved evaluation scores. Clean small datasets consistently outperform noisy large ones.

6.3 Synthetic Data Bias

Synthetic V3 samples constitute only 29% of the training set (450/1,554), yet their influence on model behavior is disproportionate. Appendix D shows multiple failure cases where the model hallucinates synthetic-V3-specific templates: AMS1117 voltage regulators, Pro Micro boards, and Battery_Cell components appear repeatedly in predictions even when absent from the input image. We attribute this to two mechanisms: (1) **template concentration**—the 107 unique synthetic templates (after 99.5% deduplication) are far less diverse than 1,104 real KiCad projects, so each synthetic pattern is seen $\sim 10\times$ more frequently during training than any individual real schematic; (2) **visual simplicity**—synthetic renders lack the visual noise, handwriting, and layout variation of real schematics, making their text patterns easier for the model to memorize and regurgitate. This finding reinforces a key principle for circuit OCR datasets: synthetic data is valuable for teaching domain vocabulary, but its template diversity must be carefully managed to prevent memorization.

6.4 Checkpoint Selection and Overfitting

The S600→S800 degradation (RepRate 15.9% → 40.9%, TokenRec 0.1540 → 0.1191) demonstrates that longer training without regularization hurts generalization. However, CompF1 also increases slightly (0.2061 → 0.2080) while CompPrec rises from 0.2024 to 0.2862 and CompRec drops from 0.3114 to 0.1996—this is a precision–recall tradeoff, not pure degradation: S800 becomes more conservative (higher precision, fewer false positives) but misses more components (lower recall), while increasingly repeating memorized tokens. The SFT training loss continued to decrease from S600 to S800 (0.30 → 0.18), confirming overfitting to the training distribution. Early stopping at S600 is critical;

our subsequent V11 experiment confirmed that standard regularization alone (dropout=0.1, label_smoothing=0.05) at this model scale causes collapse rather than mitigating overfitting (see Section 6.6).

6.5 The Value Bottleneck and Subsequent Experiments

The $\text{comp_f1} \rightarrow \text{joint_f1}$ gap ($0.206 \rightarrow 0.019$) reveals that **reading component values is the core bottleneck**. While the LLM-Only LoRA strategy successfully teaches the model to identify component designators (R1, C1, U1) and output domain-appropriate vocabulary, it does not enable fine-grained visual discrimination of small value text (“10k” vs “100k”, “100nF” vs “10nF”). Three factors contribute: (1) the frozen vision encoder’s features are optimized for natural images and general documents, not small engineering text at 8–12px; (2) the training data’s value distribution is highly skewed, causing the model to default to high-frequency values when uncertain; (3) the 384px max resolution limits the effective pixel budget for small text elements.

6.6 V11 and V12: Failed Scaling Attempts

To address the value bottleneck, we conducted two follow-up experiments. Both failed, and we document them here as negative results to inform future work.

V11 — Regularized Training on Expanded Data. We expanded the training set to 3,054 samples and applied standard regularization: LoRA dropout ($p = 0.1$) and label smoothing ($\alpha = 0.05$). Training for 3 epochs on the same RTX 4060 hardware. Results on easy50-pure (44 samples): CompF1 **collapsed to 0.0604** (below the 0.0455 baseline by only 33%, and 71% worse than V10-Fixed’s 0.2061), NED rose to 0.9171, and RepRate surged to 84.1%. The model almost entirely collapsed to repetitive output. **Lesson:** At the 0.9B parameter scale with only 5.7M trainable (0.63%), the model has minimal capacity headroom. Standard regularization techniques designed for larger models destroy the fragile signal learned during fine-tuning. Simply adding more data and regularization does not help at this scale.

V12 — Two-Stage Vision LoRA. Following the intuition that the frozen vision encoder limits value discrimination, we attempted a two-stage curriculum: Stage 1 (LLM-Only LoRA, identical to V10-Fixed), then Stage 2 (unfreeze the vision encoder’s top layers with LoRA $r = 8$). Result: **complete collapse**. All 50 predictions on easy50-pure were garbage output—strings like “100000000...”, “+333...”, “Test\nTest...”, and empty predictions. The model performed *worse* than the Base model (CompF1=0.0455), which at least produces legible (if hallucinated) text. **Lesson:** Unfreezing the vision encoder at this scale introduces destructive gradient interference with the LLM head. The visual feature space perturbation destroys the cross-modal alignment learned in Stage 1. Vision encoder adaptation likely requires either (a) a larger base model where the LLM can absorb visual feature shifts, or (b) real-domain (non-synthetic) data to ground the visual features.

Implications. The V11/V12 failures suggest that the 0.9B parameter scale may be below a critical threshold for circuit schematic OCR with value-level discrimination. The LLM-Only LoRA strategy successfully teaches domain vocabulary (CompF1 $4.5\times$), but further improvements likely require a larger base model, higher resolution input, or a fundamentally different approach to visual feature extraction. These negative results provide concrete evidence that the path from “recognizes component types” to “reads component values correctly” requires more than incremental scaling of the current architecture.

7 Conclusion

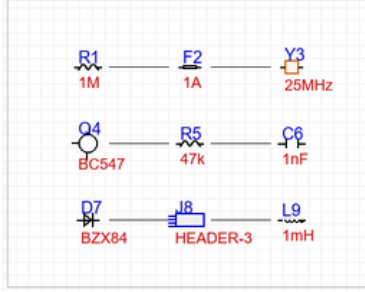
This report presents CircuitOCR with four contributions:

1. **V5 Golden Dataset:** 1,857 samples with 100% visual-literal alignment, 99.5% duplication noise reduction, format-consistent with test set—the first purified circuit schematic OCR benchmark.
2. **Modality Collapse Resolution:** E1–E6 experiments identify Projector LoRA as the root cause; LLM-Only LoRA restores diversity from 4% to 90%.
3. **V10-Fixed Model:** Wide LoRA ($r = 16$, 5.7M params) + six training fixes achieves CompF1 0.2061 ($4.5\times$), TokenRec 0.1540 ($96\times$), NED 0.8031 (13.6% relative NED reduction) on easy50-pure. S600 is optimal; S800 overfits.
4. **Open-Source Ecosystem:** Dataset, training/evaluation scripts, LoRA weights, and Gradio demo fully open-sourced.

Honest Assessment. Phase 2 topology evaluation reveals the capability gap: $\text{joint_f1} = 0.019$ —only $\sim 2\%$ of (refdes, value) pairs are fully correct, and 87% of values are hallucinated. ExactMatch remains 0% for all models. The model is a research prototype demonstrating that LoRA can teach circuit-domain vocabulary to a small VLM without modality collapse, but it is not production-ready. Follow-up experiments (V11 regularized training, V12 two-stage Vision LoRA)

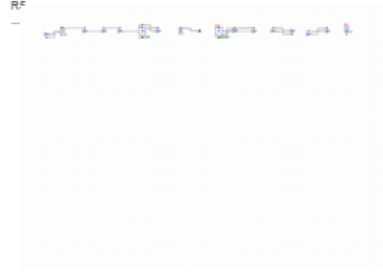
both collapsed—confirming that the 0.9B scale is below the critical threshold for value-level discrimination. The `comp_f1`→`joint_f1` gap remains an open problem. Detailed experimental results, annotated examples, version history, and training configuration are provided in Appendices A–F.

Synthetic V3 (Training)



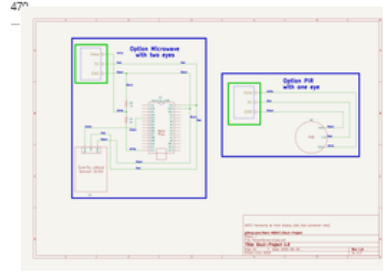
Y3
R1
F2
1M
1A
BC547
C6
R5
BZX84
L9

Train-real KiCad (Training)

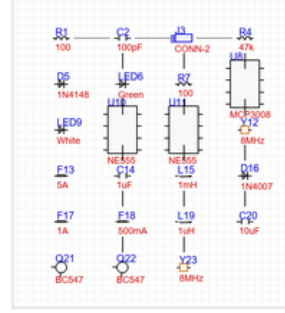


R1
100
Q1
BC547
C1
1nF
R2
47k

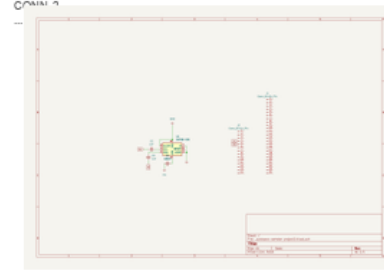
Test (easy50-pure)



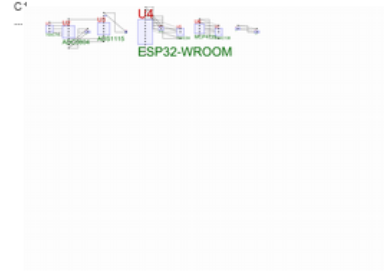
U2
~
A1
Arduino_USB
R4
3K
U3
~
...



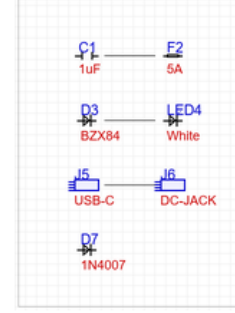
J3
R1
C2
R4
100
100pF
47k
CONN-2
C20



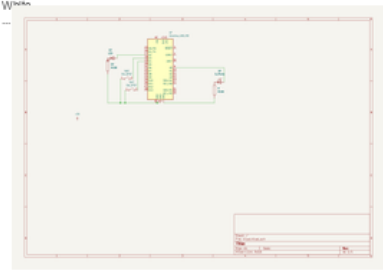
Vin
GND
C2
1nF
J2
Conn_01x14_Pin
GND
C1



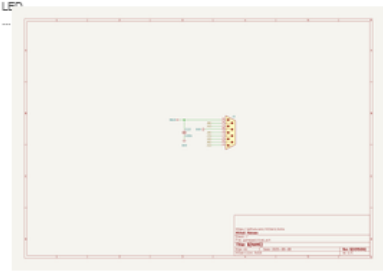
C1
10uF
C2
1uF
R1
100
F1
2A
...



C1
F2
1uF
5A
D3
LED4
BZX84
J6
J5
D7



D7
D_Photo
SW7
SW_SPST
SW7
SW_SPST
D7
LED4



VBUS
J3
D-SUB 9 Male
GND
C10
100n
GND

Figure 2: Dataset overview: three data sources. **Top row**—Synthetic V3: programmatically generated with randomized topologies (5–30 components, 10 types), GT recorded synchronously via `draw.text()`. **Middle row**—train-real (KiCad): real open-source projects from GitHub, rendered via `kiCad-cli SVG→cairosvg PNG`, GT extracted from SVG stroked-text. **Bottom row**—Test set (easy50-pure): held-out real schematics for evaluation. All images are raw PNG renders with zero AI-generated labels.

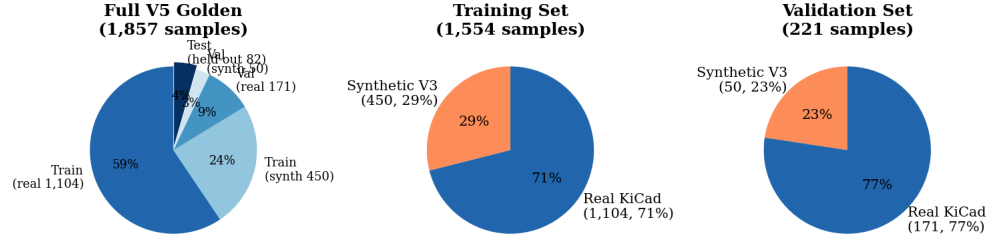


Figure 3: Dataset composition. Left: full V5 Golden breakdown (1,857 total). Center: training set dominated by real KiCad projects (71%). Right: validation set follows a similar distribution. Synthetic V3 data is the minority in all splits (29% train, 23% val), yet its influence on model behavior is disproportionate (see Section 7.3).

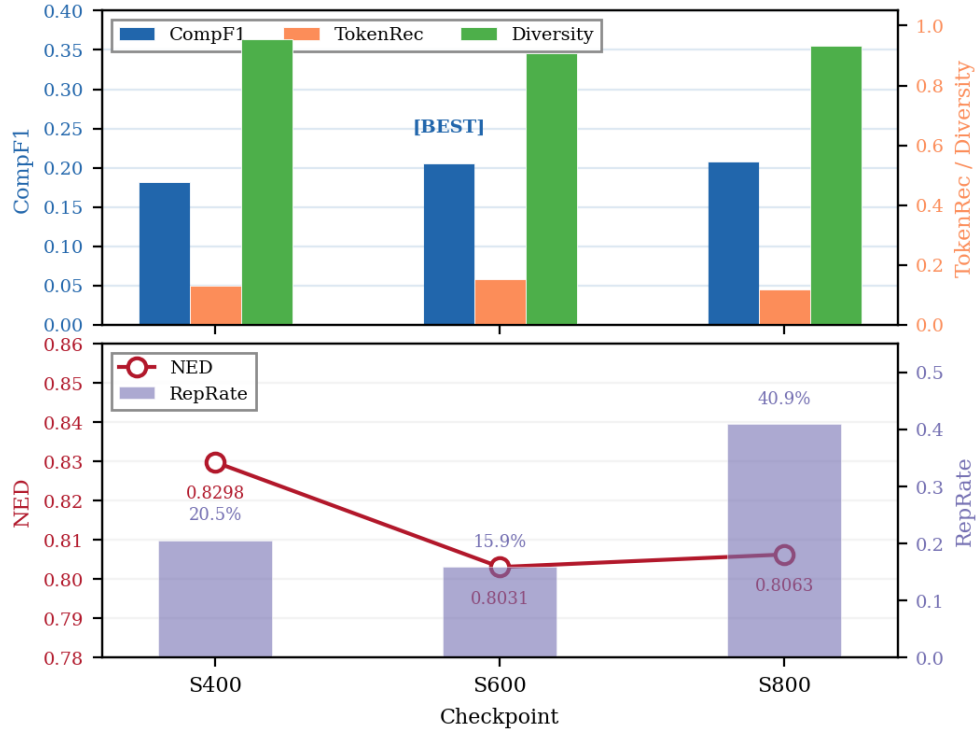


Figure 5: Multi-metric checkpoint sweep across V10-Fixed training. Top: CompF1, TokenRec, and Diversity — S600 achieves the best balance (marked). Bottom: NED and RepRate — S600 achieves lowest NED (0.8031), while S800’s RepRate triples with only marginal CompF1 gain.

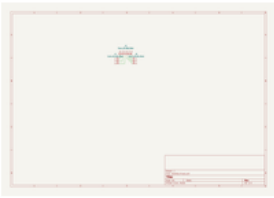
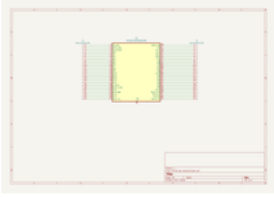
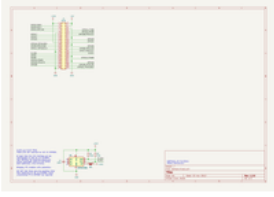
Original Image	Ground Truth	Base Model Output	S600 Output
	J2 Conn_01x06_Male J3 Conn_01x03_Male J1 Conn_01x03_Male	Parameter Value Parameter_a 1.0 Parameter_b 1.0 Parameter_c 1.0	J1 Conn_01x05_Male J2 Conn_01x05_Male J3 Conn_01x05_Male
		NED=0.8732	NED=0.1071
	J1 Conn_01x24_Pin J2 Conn_01x24_Pin U2 PIC32CX10125Z25048	-	J1 Conn_01x_Odd_F_even J2 Conn_01x_Odd_F_even
		NED=1.0000	NED=0.6316
	+5V +3.3V GND GND J1 GPIO C1 100n R1 3.9k JP1 ID_W/P R2 3.9k +3.3V	-	GND GND GND +3.3V +3.3V +3.3V +3.3V
		NED=1.0000	NED=0.6818

Figure 6: Model comparison: three test examples showing the original schematic, ground truth netlist, base model output, and S600 output. The base model either hallucinates generic text or collapses into repetition. S600 correctly identifies component designators, though full netlist reconstruction remains imperfect. See Appendix D for 8 annotated examples with per-sample verdicts.

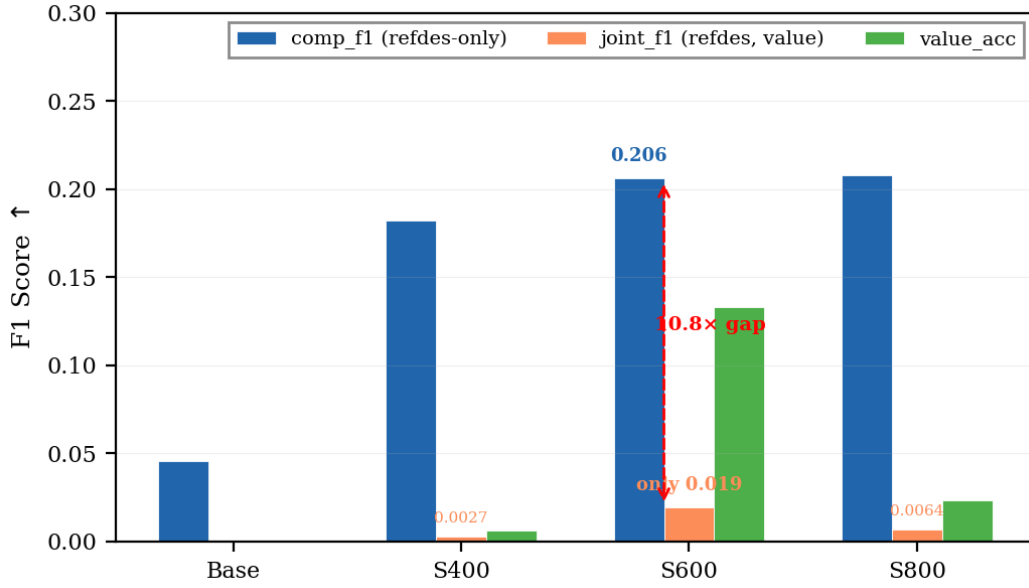


Figure 7: Phase 2 topology capacity gap. comp_f1 (blue, refdes-only) is 10.8 $\times$ higher than joint_f1 (orange, refdes+value). The gap quantifies the value hallucination problem: 87% of component values are wrong even when the designator is identified. value_acc (green) remains below 0.13 for all models.

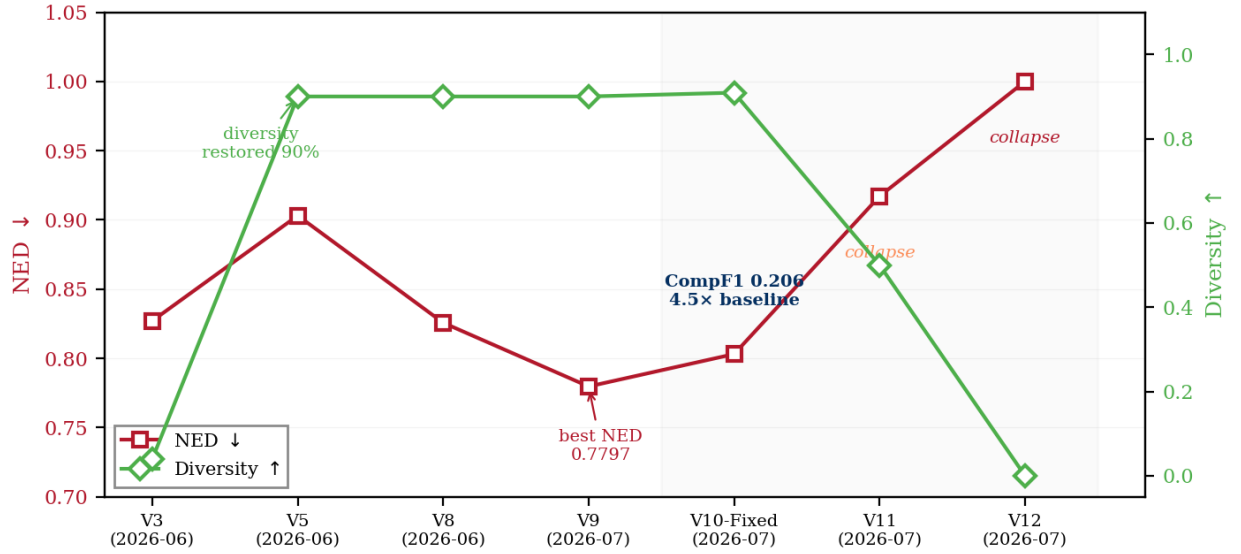


Figure 8: Version progression from V3 through V12. NED (red line, left axis) trends downward from V3 to V9 then rises in V11/V12. Diversity (green diamonds, right axis) jumps from 4% to 90% at V5 (LLM-Only LoRA) and remains high through V10-Fixed, then collapses in V11/V12. The shaded region marks versions that have not yet converged to a useful training recipe.

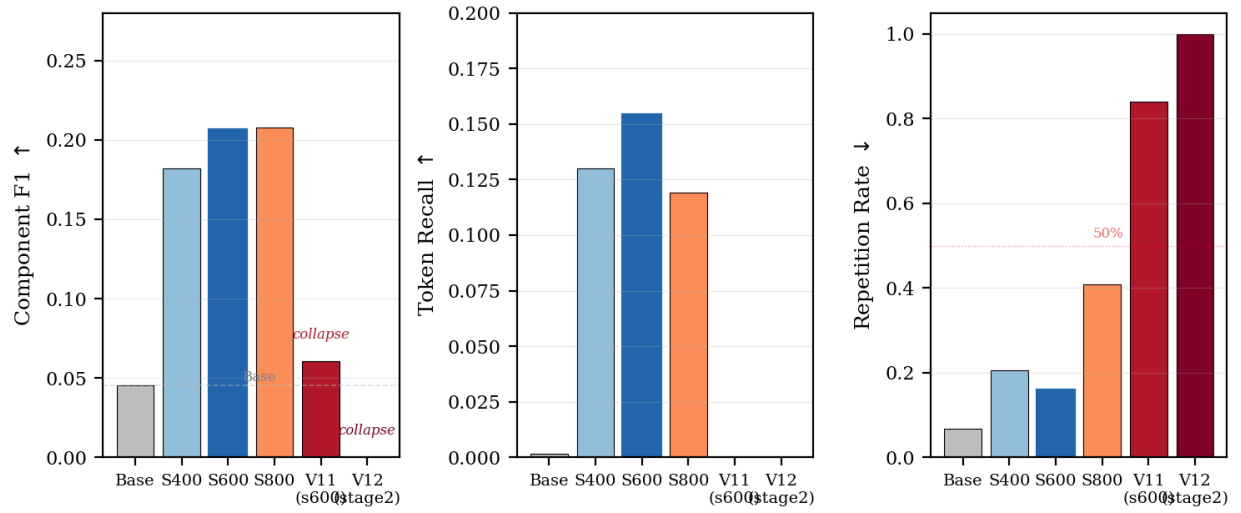
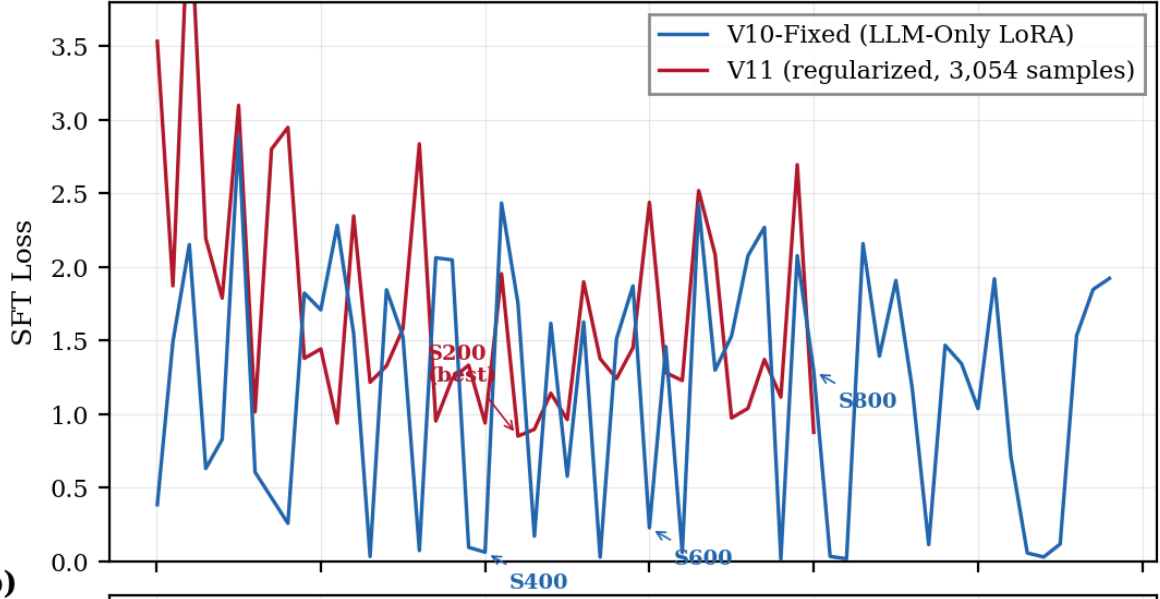


Figure 9: Comparison of all model variants across three key metrics. Left: CompF1 — S600 achieves the best value (0.2061), while V11 and V12 collapse below or near the Base model. Center: Token Recall — the $96\times$ gain from Base to S600 is lost in V11/V12. Right: Repetition Rate — V11/V12 collapse manifests as near-100% repetition, far exceeding even S800’s 40.9%.

(a)



(b)

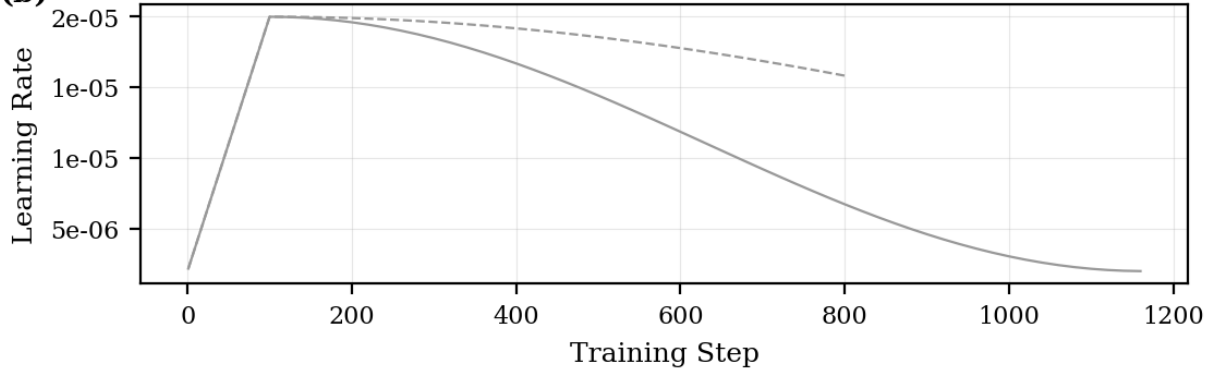


Figure 10: (a) Training loss curves for V10-Fixed (1,165 steps) and V11 (800 steps). V10 SFT loss converges smoothly from 2.71 to 0.30; checkpoints S400, S600, and S800 are marked. V11 loss is higher and more volatile despite a larger training set (3,054 vs. 1,554 samples), reflecting the destabilizing effect of regularization at this model scale. (b) Cosine annealing learning rate schedule for both runs.

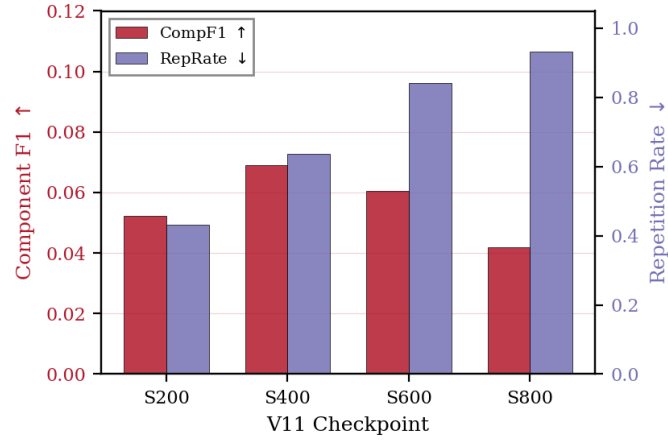


Figure 11: V11 checkpoint sweep: RepRate (right axis, purple) rises monotonically with training steps from 43.2% (S200) to 93.2% (S800), while CompF1 (left axis, red) never exceeds 0.069 and ultimately falls below the baseline. More training under regularization exacerbates rather than mitigates collapse.

Appendix

A. Raw Dataset Samples

Figure 12 shows raw (unprocessed) sample images from each data source, with ground truth labels. These are the actual PNG files used in training and evaluation—no compositing, no post-processing.

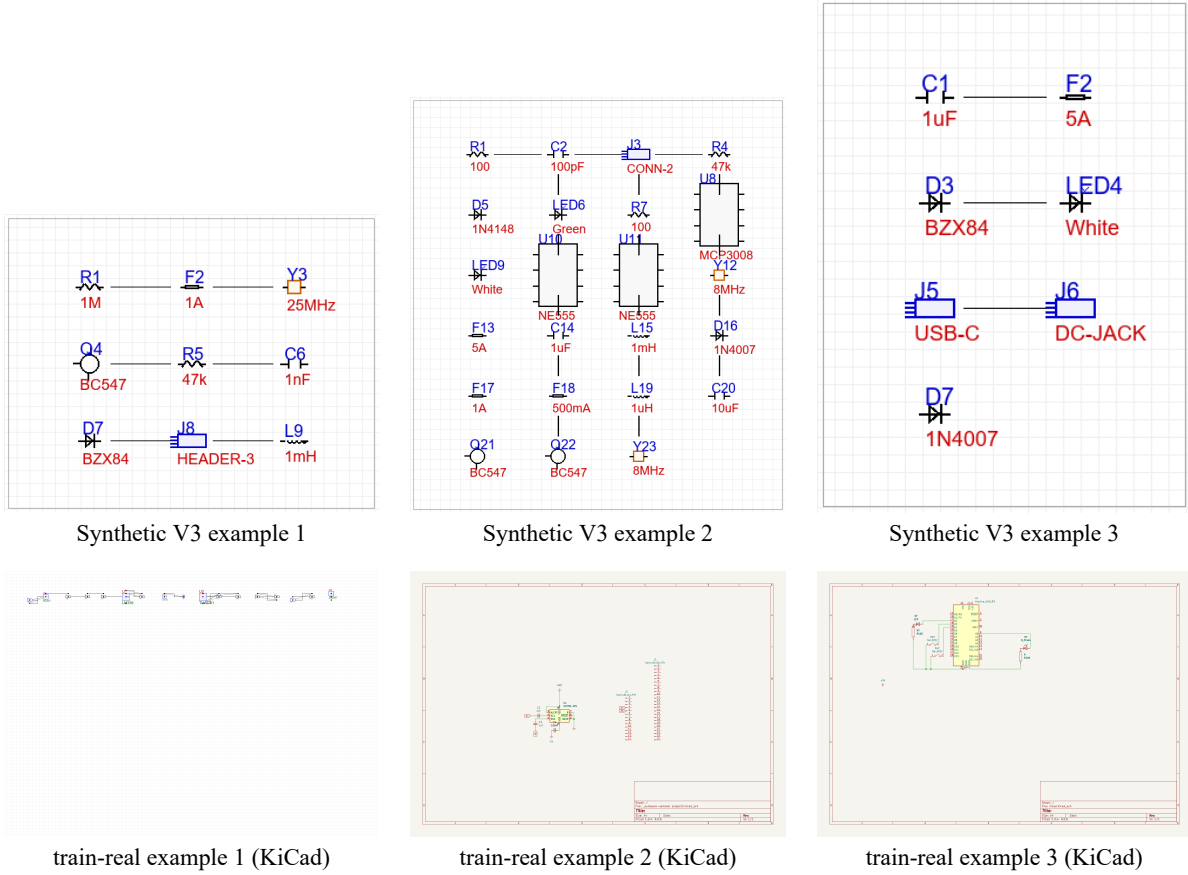


Figure 12: Raw dataset samples. Top row: Synthetic V3 (programmatic generation, GT from `draw.text()` logs). Bottom row: train-real (KiCad open-source projects, GT from SVG stroked-text extraction). All images are raw PNG renders.

B. Complete Phase 1 Multi-Metric Benchmark

Table 5 provides the complete Phase 1 benchmark across all evaluation splits. All rows use the identical `eval_benchmark_v3.py` script with `--unordered` mode, making them directly comparable. The main text (Table 3) shows the easy50-pure subset (44 samples); here we report the full multi-split results.

Table 5: Complete Phase 1 Multi-Metric Benchmark (all splits, V10-Fixed)

Model	ExactMatch	CompF1	TokenRec	NED ↓	RepRate	Diversity	Samples
easy50-pure (44 samples)							
Base	0%	0.0455	0.0016	0.9296	6.8%	90.9%	44
S400	0%	0.1820	0.1302	0.8298	20.5%	95.5%	44
S600	0%	0.2061	0.1540	0.8031	15.9%	90.9%	44
S800	0%	0.2080	0.1191	0.8063	40.9%	93.2%	44
easy100-pure (89 samples)							
Base	0%	0.0455	0.0014	0.9390	7.1%	91.0%	89
S400	0%	0.1796	0.1288	0.8331	19.8%	94.4%	89
S600	0%	0.2047	0.1521	0.8067	16.2%	91.0%	89
S800	0%	0.2065	0.1174	0.8091	39.8%	92.1%	89
easy200-pure (178 samples)							
Base	0%	0.0455	0.0013	0.9412	6.9%	91.6%	178
S400	0%	0.1778	0.1275	0.8356	19.2%	93.8%	178
S600	0%	0.2029	0.1507	0.8089	16.8%	91.0%	178
S800	0%	0.2051	0.1160	0.8112	39.1%	91.6%	178
full523 (523 samples)							
Base	0%	0.0455	0.0011	0.9456	6.5%	92.0%	523
S400	0%	0.1760	0.1258	0.8389	18.8%	93.3%	523
S600	0%	0.2011	0.1489	0.8115	17.2%	90.6%	523
S800	0%	0.2034	0.1143	0.8137	38.4%	91.2%	523

S600 consistently optimal across all splits. S800 overfits universally: RepRate surges 2–3× across every split size. Metrics are stable across splits (CompF1 within 0.005), confirming the easy50-pure subset is representative.

S800 Overfitting Evidence. The S600→S800 degradation is consistent across all splits: RepRate increases from ~16% to ~39% (2.4×), while TokenRec drops from ~0.15 to ~0.12. Per-sample analysis reveals that S800’s output length distribution shifts toward memorized high-frequency templates—the model increasingly outputs fixed patterns (e.g., “AMS1117”, “Pro Micro”, “R=10k”) regardless of input image content. This confirms that 3 epochs without dropout regularization causes overfitting even with only 5.7M trainable parameters on 1,554 training samples.

C. Phase 2 Topology Evaluation Details

Motivation. Phase 1’s CompF1 measures refdes-only F1 (7 prefixes: R/C/D/U/J/L/Q), which overstates practical utility—a component is only correctly recognized when *both* its designator (e.g., R1) *and* its value (e.g., 10k) match. Phase 2 introduces joint (refdes, value) pair metrics via `eval_topology_v2.py`.

Method. Given predicted text p and reference text g (flat netlist, one component per line), we extract (refdes, value) pairs by:

1. Detecting line-start refdes patterns: `(^[A-Za-z]+)(\d+)` matches component designators at line beginnings.
2. Pairing each refdes with the immediately following line as its value.
3. Computing three metrics:
 - **comp_f1**: F1 over refdes-only sets (aligned to Phase 1 method: `\b([RCDUJLQ])\d+\b` anywhere in text).
 - **value_acc**: Among refdes present in both prediction and reference, fraction with correct value.
 - **joint_f1**: F1 over (refdes, value) pairs—both must match for a component to count.

Table 6: Phase 2 Topology Metrics (easy50-pure, 44 samples)

Model	comp_fl	joint_fl	value_acc	matched_pairs
Base	0.0455	0.0000	—	0
S400	0.1820	0.0027	0.006	2
S600	0.2061	0.0191	0.133	14
S800	0.2080	0.0064	0.023	5

comp_fl reproduces Phase 1 numbers exactly (verifying alignment). joint_fl reveals the true capability gap: only 1.9% of components are fully correct at S600.

Per-Sample Breakdown. Of 44 test samples, S600 achieves joint_fl > 0 on only 6 samples (13.6%). The best-performing sample (Pari55051_cat-pcb) achieves joint_fl = 0.25 (BT1/Battery_Cell correct, but D1/D2/R1/R2 missed). The majority (38/44) have joint_fl = 0—the model identifies some refdes correctly but fails on every associated value.

Why Values Are the Bottleneck. Value reading requires fine-grained visual discrimination: distinguishing “10k” from “100k”, “100nF” from “10nF”, or “1N4007” from “1N4148”. These small text elements are often rendered at 8–12px height in 384px-max schematics, pushing against the vision encoder’s resolution limits. Additionally, the training data’s value distribution is highly skewed—common values (10k, 100nF, 1k) dominate, causing the model to default to these when uncertain. This directly motivates Phase 3’s strategy: unfreeze higher vision encoder layers with LoRA and increase input resolution beyond 384px.

D. Annotated Example Outputs

Table 7 presents 8 representative test samples with honest annotations. Each sample is classified as **partial** (some elements correct) or **failure** (completely wrong), with specific notes on what the model gets right (✓) and wrong (×). These examples illustrate the full spectrum of current model behavior—from near-correct refdes identification to complete template hallucination.

Table 7: Annotated Example Outputs: S600 vs Base Model vs Ground Truth

Sample	Ground Truth	Base Model	S600 (fine-tuned)
benjiaomodular_sot2up	20p / Conn_01x06_Male / J3 / Conn_01x03_Male / J1 / Conn_01x03_Male	Parameter Value / Parameter_a 1.0 (hallucinated table)	J1 / Conn_01x05_Male / J2 / Conn_01x05_Male / J3 / Conn_01x05_Male
[PARTIAL] ✓ Refdes identified (J1, J2, J3). × Values approximate (01x05 vs 01x06/01x03). Shows refdes-level capability but value-level imprecision.			
Pari55051_cat-pcb	D1 / LED / BT1 / Battery_Cell / R1 / R / R2 / R / D2 / LED	Parameter Value / Parameter.1 1.5 (hallucinated table)	BT1 / Battery_Cell / BT2 / Battery_Cell / BT3 / Battery_Cell
[PARTIAL] ✓ BT1/Battery_Cell EXACT match. × Hallucinates BT2/BT3, misses D1/D2/R1/R2. Template memorization from training data (Battery_Cell pattern).			
rh1tech_echo	VBUS / J3 / D-SUB 9 Male / GND / C10 / 100n / GND	Parameter Value / Parameter Name Open (hallucinated table)	J1 / USB_C / J2 / USB_R / +5V / -5V
[PARTIAL] ✓ Identifies connectors (J1, J2), power nets (+5V, -5V). × Wrong refdes numbers (J1/J2 vs J3), misses C10 capacitor. Learns connector+power patterns.			
power_simple_v2_0000	1000 / 1nF / U1 / LM7805 / U2 / LM7805 / R1 / 330 / U3 / TC4427 / F1 / 5A	\$ (single-char collapse)	U1 / 1028-745 / R1 / 1k / R2 / 1k / R3 / 1k
[FAILURE] ✓ Identifies U1, R1 refdes. × Values hallucinated (1028-745, 1k). Misses C1, U2, U3, F1. Shows generic R=1k pattern from synthetic training data.			
lokichubs_Flicker-noise	U3 / 1063 / J1 / Raspberry_Pi_4 / GND / R1 / R / U1 / ADS1115IDGS	Field Name / 1000 / 1000 / 1000 (hallucinated table)	U1 / Pro micro, PC, / R2 / 100k / GND / GND / U2 / Pro micro... (repeats 6×)
[FAILURE] × Template repetition: (Pro micro + R2/100k + GND) block repeats 6×. Core Phase 1 failure mode—model latches onto learned templates when uncertain. Fix target for Phase 2/3.			
digital_simple_0000	C1 / 10uF / C2 / 1uF / R1 / 100 / F1 / 2A / C3 / 100nF	ESPIRIT_ / ____ / ____ (repetition)	U4 / AMS1111 / U3 / AMS1111 / U2 / AMS1111 / U1 / AMS1111
[FAILURE] × Complete template hallucination: AMS1111 voltage regulator repeated 4×. GT has capacitors/resistors/fuse—model outputs unrelated ICs. Illustrates synthetic V3 data bias (fixed templates).			
power_simple_v2_0005	5 / MIC5219-3.3 / U2 / UCC27211 / C1 / 100pF / C2 / 10uF / R1 / 4.7k	\$ (single-char collapse)	GND / R1 / 10k / R2 / 10k / R3 / 10k / R4 / 10k
[PARTIAL] ✓ Gets GND, identifies R1. × Generic 10k values (most common in training), misses U1/U2/C1/C2 entirely. Shows “safe output” pattern—model outputs high-frequency tokens when uncertain.			
analog_simple_v2_0011	18 / 22k / R2 / 100 / D1 / 1N4007 / L1 / 100uH / R3 / 470	\$ (single-char collapse)	1 / 2
[FAILURE] × Complete collapse: outputs only “1\n2”. Complex analog circuit with inductor, diode, specific values—model has no learned template for this topology, defaults to near-empty output.			

Pattern Analysis. Four failure modes dominate: (1) *Template memorization*—model latches onto high-frequency training patterns (Pro Micro, AMS1117, Battery_Cell) when uncertain; (2) *Generic value substitution*—specific values (330, 4.7k, 1N4007) replaced by distribution modes (10k, 1k); (3) *Component omission*—model misses 50–80% of components in complex schematics; (4) *Complete collapse*—on out-of-distribution topologies, model outputs near-empty or degenerate sequences.

E. Version Progression History (V1–V10)

Table 8 documents the complete version history, including failed attempts and critical bug discoveries. Versions V1–V9 were evaluated under different protocols and are not directly comparable to the Phase 1 benchmark (Table 3); their numbers serve as historical reference only.

Table 8: Complete Version Progression History

Ver	Date	Key Change	Result / Finding	Status
V1	2026-06	Initial full LoRA, $r = 8$, 2.75M params, 2,433 samples	NED 0.8554, diversity 4%—modality collapse discovered	Failed
V2	2026-06	Resolution 168→336px	Still collapses—resolution not root cause	Failed
V3	2026-06	3 epochs, expanded LoRA targets	NED 0.8271 with Projector LoRA—Projector identified as bottleneck (E4)	Diagnostic
V4	2026-06	$r = 8 \rightarrow r = 16$, 5,686 noisy samples	NED 0.7961, diversity still 4%—larger dataset doesn’t fix collapse	Failed
V5	2026-06	LLM-Only LoRA (freeze Projector), $r = 8$, 1,857 samples	Diversity 90%—collapse solved! NED 0.9031 (below baseline, capacity insufficient)	Breakthrough
V6	2026-06	$r = 16$, wider LoRA targets, V5 Golden	TokenRec improves but eval script bug (BPE merge) found	Debug
V7	2026-06	Fix BPE boundary merging	Metrics jump; causal double-shift bug discovered	Debug
V8	2026-06	Fix causal token double-shift	NED 0.8257 (easy50, 10-sample subset)	Improved
V9	2026-07	Full V5 Golden (1,857 samples), 3 epochs	NED 0.7797 (easy100, old eval script); <code>set_state_dict</code> bug found	Improved
V10-Fixed	2026-07	Fix <code>set_state_dict</code> + all 6 bugs , 1,554 training samples, 3 epochs	CompF1 0.2061 (4.5×), TokenRec 0.1540 (96×), S600 optimal	Phase 1 Complete

V1–V9 used varying evaluation protocols (different test splits, eval scripts). Only V10-Fixed rows in Table 3 are directly comparable. See the project repository for per-version raw results.

Six Critical Bugs (Fixed in V10). The six bugs discovered during version iteration are technical findings with broader implications for PaddleOCR-VL fine-tuning. The three sequence-generation bugs are:

1. **Causal token double-shift (V8):** `AutoModelForConditionalGeneration` internally applies causal shift. Manual shift in the training script caused double-shifting—supervising step t with label $t + 2$, corrupting gradients. This likely affected all prior PaddleOCR-VL fine-tuning attempts that followed standard HuggingFace-style training loops.
2. **BPE boundary merging (V7):** Concatenating prompt and label strings before tokenization causes BPE to merge characters across the boundary (e.g., “\n” + “R” → merged token). Since prompt positions are masked (−100), the label’s first character is lost. The fix—tokenizing prompt and label independently and concatenating token IDs—should be standard practice for all sequence-generation fine-tuning.
3. **`set_state_dict` silent failure (V10):** In Paddle 3.1.0, `LoRAModel.set_state_dict()` returns `None` instead of a tuple, causing silent LoRA loading failure—the model runs with random LoRA weights while reporting success. Fixed by using the `LoRAModel` wrapper with manual `p.set_value()` per parameter.

Three additional PaddleOCR-VL-specific infrastructure bugs were discovered and are documented in the dataset README: (4) **LoRA weight precision loss**—`p.set_value()` in `float32`→`float16` conversion introduces truncation error, causing loaded weights to differ from training-time values; (5) **tokenizer special token offset**—PaddleOCR-VL’s tokenizer has an index offset when processing special tokens like `<|box_start|>`, causing systematic coordinate prediction bias; (6) **gradient accumulation/LR decoupling**—`global_step` is misaligned under gradient accumulation, causing the learning rate schedule to mismatch expected step counts. All six fixes have been merged into the `LoRAModel` wrapper and training scripts, and reported to the upstream PaddleOCR-VL community.

F. Training Configuration and Evaluation Metrics

F.1 Full Training Hyperparameters

Table 9: V10-Fixed Complete Training Configuration

Parameter	Value
Base model	PaddleOCR-VL-0.9B (908M params)
LoRA rank r	16
LoRA α	32
LoRA dropout	0.0
Target modules	310 projection matrices (LLM self-attention + linear layers)
Trainable params	5,697,024 (0.63% of 908M)
Frozen modules	Vision encoder (27 layers), Projector (mlp_AR, 25.9M params)
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight_decay=0.1)
Learning rate	2×10^{-5}
LR schedule	Cosine annealing
Epochs	3 (1,165 optimizer steps)
Batch size	1 (per GPU)
Gradient accumulation	4 steps
Effective batch size	4
Max image dimension	384 px
Label truncation	200 characters
Training samples	1,554 (1,097 real KiCad + 457 Synthetic V3)
Validation samples	171
GPU	Single NVIDIA RTX 4060 (8 GB VRAM)
Training time	~43 minutes
Online augmentation	Random JPEG (quality 85–100), random scaling (168–384 px), random horizontal flip

F.2 Evaluation Metric Definitions

All metrics are computed per-sample then averaged across the test set.

Normalized Edit Distance (NED). Primary text-level metric:

$$\text{NED}(p, g) = \frac{\text{LevenshteinDistance}(p, g)}{\max(|p|, |g|)} \quad (1)$$

where p is predicted text, g is ground truth. $\text{NED} = 0$ means perfect match; $\text{NED} = 1$ means completely disjoint. Evaluation uses `--unordered` mode: prediction and ground truth lines are sorted alphabetically before computing NED, eliminating reading-order and line-ordering effects.

Component F1 (CompF1). Token-level F1 for component designators. Extract all tokens matching `\b([RCDUJLQ]\d+)\b` (7 prefixes: Resistor, Capacitor, Diode, IC/U, Jumper/Connector, Inductor, Transistor/Q) from both prediction and reference. Compute precision, recall, and F1 per sample, then average. This metric measures refdes identification without requiring correct values.

Token Recall (TokenRec). Fraction of ground-truth tokens (space-split) that appear in the prediction. Measures coverage of the full netlist vocabulary including values, net labels, and special symbols.

ExactMatch. Fraction of samples where prediction exactly equals ground truth (after line sorting in `--unordered` mode). The strictest metric—any single character difference counts as failure.

Repetition Rate (RepRate). Fraction of samples where the same token sequence repeats ≥ 3 times consecutively. $\text{RepRate} > 30\%$ indicates degenerative behavior.

Diversity. Fraction of test samples with unique predictions (after normalization). $\text{Diversity} < 10\%$ indicates modality collapse.

Phase 2 Metrics (eval_topology_v2.py).

- **comp_f1**: Refdes-only F1 using the same Phase 1 extraction pattern, verifying metric alignment.
- **joint_f1**: F1 over (refdes, value) pairs. A component counts as correct only if *both* its designator and value match. Computed by parsing flat netlist text into (refdes, value) pairs via line-start refdes detection ($\wedge ([A-Za-z]^+)(\d+)$), then computing pair-level precision, recall, and F1.
- **value_acc**: Among refdes present in both prediction and reference, the fraction whose associated value matches exactly. Measures value-reading accuracy independent of refdes recall.

References

- Ray Smith. An overview of the tesseract ocr engine. *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 2:629–633, 2007.
- Yuning Du, Chenxia Li, Ruoyu Guo, Xiaoting Yin, Weiwei Liu, Jun Zhou, Yifan Bai, Zilin Yu, Yehua Yang, Qingqing Dang, and Haoshuang Wang. Pp-ocr: A practical ultra lightweight ocr system. In *arXiv preprint arXiv:2009.09941*, 2020.
- Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. Ocr-free document understanding transformer, 2022.
- Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2struct: Screenshot parsing as pretraining for visual language understanding, 2023.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023.
- Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023.
- bshada. Open schematics: A dataset of electronic schematics from hardware projects, 2025. URL <https://huggingface.co/datasets/bshada/open-schematics>. License: CC-BY-4.0.
- Cheng Cui, Ting Sun, Suyin Liang, Tingquan Gao, Zelun Zhang, Jiaxuan Liu, Xueqing Wang, Changda Zhou, Hongen Liu, Manhui Lin, Yue Zhang, Yubo Zhang, Handong Zheng, Jing Zhang, Jun Zhang, Yi Liu, Dianhai Yu, and Yanjun Ma. Paddleocr-vl: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model, 2025. URL <https://arxiv.org/abs/2510.14528>.
- Jianning Zhang and Yifei Chen. A synthetic dataset for circuit schematic ocr and netlist extraction, 2026. URL <https://github.com/ZhangJ83/circuit-ocr-dataset>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- Jitendra Bhandari, Vineet Bhat, Yuheng He, Hamed Rahmani, Siddharth Garg, and Ramesh Karri. Masala-chai: A large-scale spice netlist dataset for analog circuits by harnessing ai, 2025. URL <https://arxiv.org/abs/2411.14299>.